# The local identifying code number of the six dimensional binary cube

GPT-6 Astra research project
AI-assisted research manuscript

11 October 2026

**Abstract**

A local identifying code is a set of vertices that meets every closed neighborhood and gives different neighborhood intersections to adjacent vertices. We give a 14-vertex local identifying code in the six-dimensional binary cube and an exact, independently checked propositional certificate excluding every code of size at most 13. The lower bound combines a short structural reduction with 64 domination clauses, 192 separation clauses, a sequential cardinality counter, and a reverse-unit-propagation refutation. This establishes the value $M^{\mathrm{L}}(6) = 14$, improving the previously located interval $12 \le M^{\mathrm{L}}(6) \le 15$. A Lean 4 development formalizes the witness, graph encoding, cardinality extension, symmetry reduction, and all 61 970 RUP steps. The unconditional exact-minimum theorem has passed both a complete kernel build and an independent clean-source rebuild, using only standard axioms. The manuscript specifies the reproducible verification procedure, certificate hashes, and trust boundary of each form of evidence.

**Verification record.** All 61 970 RUP steps and the unconditional exact-minimum theorem passed both the initial Lean kernel build and an independent clean-source rebuild of 257 modules. The final theorem reports only `propext`, `Classical.choice`, and `Quot.sound`. The project label in the author line does not certify runtime model identity. No human peer review or global publication priority is asserted.

## 1 Definitions and result

Let $Q_n$ be the graph with vertex set $\{0,1\}^n$, where two vertices are adjacent if their Hamming distance is one. Its closed radius-one ball is

$$B_1(v) = \{v\} \cup \{v \oplus e_i : 0 \le i < n\},$$

where $e_i$ has its sole nonzero coordinate in position $i$ and $\oplus$ is coordinatewise addition modulo two. For $C \subseteq V(Q_n)$ write $I_C(v) = B_1(v) \cap C$. Following Herva, Laihonen, and Lehtilä [1, Definition 1.3], $C$ is *local 1-identifying* if

$$I_C(v) \neq \varnothing \qquad \text{for all } v, \tag{1}$$

$$I_C(u) \neq I_C(v) \qquad \text{whenever } uv \in E(Q_n). \tag{2}$$

The minimum possible cardinality is denoted $M^{\mathrm{L}}(n)$. Crucially, condition (2) applies to adjacent code vertices too. Separating only adjacent non-code vertices would define a different parameter. No separation of arbitrary nonadjacent pairs is required.

**Theorem 1** (Exact minimum)**.** *For local 1-identifying codes in $Q_6$, $M^{\mathrm{L}}(6) = 14$. More explicitly, there exists a local 1-identifying code of cardinality 14, and every local 1-identifying code in $Q_6$ has cardinality at least 14.*

Theorem 1 has both an independently checked finite certificate argument and a complete Lean proof reproduced from a clean source snapshot. Section 7 describes the unconditional formal statement and verification records. The claim concerns exactly conditions (1)–(2), with no exclusion of edges incident to codewords.

The binary cube is a natural finite setting for local identification: one wants nonempty observations at every vertex and distinct observations across an edge. Our contribution is a small exact extremal value with inspectable computational evidence. We make no claim that this code constitutes a cryptographic primitive, improves a deployed security system, or solves a general identification problem.

## 2 Prior bounds and scope of novelty

The strongest bound located in the primary source is

$$12 \le M^{\mathrm{L}}(6) \le 15.$$

The lower bound follows from $M^{\mathrm{L}}(n) \ge 3 \cdot 2^n/(3n-2)$ in [1]. For $n = 6$, this expression equals 12. The upper bound 15 is stated explicitly in Proposition 2.9 of the published paper, printed page 362. Table 1 on printed page 358 still lists the weaker upper bound 16. The arXiv v3 PDF agrees with the journal; its experimental HTML renumbers the relevant proposition as 2.15. The correct baseline for this manuscript is therefore 15, not 16.

A targeted public-source review on 11 October 2026 checked the exact paper title, authors, identifier, and combinations of local identifying codes with hypercubes, dimension six, and cardinalities 14 and 15. It did not locate an earlier 14-word construction or an exact determination of $M^{\mathrm{L}}(6)$. This is a scoped search result, not proof of global novelty. Indexing gaps, inaccessible citation services, unpublished work, and alternate terminology remain limitations. Ordinary identifying codes, local locating-dominating codes, and linear identifying codes are not interchangeable with the present parameter.

Two elementary observations help place the witness in context. First, a binary linear or affine code has power-of-two cardinality, so a 14-word code is not itself linear or affine. Second, simply doubling a seven-word covering code of $Q_5$ is not automatically valid: if a base codeword is isolated, the two adjacent lifted copies have the same two-word signature. The published 15-word witness was checked independently, and none of its single-word deletions remains local identifying. These observations do not classify all optimal codes or exclude derivation through more complicated modifications of known constructions.

## 3 An explicit code of cardinality fourteen

Identify a binary word $b_5 \cdots b_0$ with the integer $\sum_{i=0}^{5} b_i 2^i$. Set

$$C_{14} = \{0, 4, 15, 23, 26, 29, 31, 33, 35, 36, 41, 49, 58, 62\}. \tag{3}$$

The corresponding binary words are shown in Table 1.

| Integer | Binary word | Integer | Binary word |
|---|---|---|---|
| 0 | 000000 | 33 | 100001 |
| 4 | 000100 | 35 | 100011 |
| 15 | 001111 | 36 | 100100 |
| 23 | 010111 | 41 | 101001 |
| 26 | 011010 | 49 | 110001 |
| 29 | 011101 | 58 | 111010 |
| 31 | 011111 | 62 | 111110 |

Table 1: The explicit 14-word code, with the most significant bit written first.

**Proposition 2.** *The set $C_{14}$ in* (3) *is local 1-identifying in $Q_6$.*

*Finite verification.* For each $v \in \{0, \dots, 63\}$, compute $I(v) = \{c \in C_{14} : \mathrm{popcount}(v \oplus c) \le 1\}$. All 64 sets are nonempty. For each $0 \le u < v < 64$ with $\mathrm{popcount}(u \oplus v) = 1$, check $I(u) \neq I(v)$. All 192 checks pass. Appendix A gives the complete executable check using only integer arithmetic and finite sets. The independently written checker also records a separating codeword for each edge. The Lean witness theorem uses the same mathematical definition with a separate finite decision proof checked by the kernel. □

The signature-size distribution is 38 vertices with one codeword, 20 with two, four with three, and two with four. As a consistency check,

$$38 + 2 \cdot 20 + 3 \cdot 4 + 4 \cdot 2 = 98 = 7|C_{14}|,$$

because each codeword lies in exactly seven closed balls. The verification includes the ten edges with both endpoints in $C_{14}$; in total, 74 tested edges have at least one code endpoint. Those checks matter because omitting them would silently weaken the problem.

## 4 A structural reduction for small codes

The normalization used by the shorter certificate is stronger than merely assuming $0 \in C$. We justify it without relying on solver behavior.

**Lemma 3.** *If $C$ is an independent local 1-identifying code in $Q_6$, then $|C| \ge 16$.*

*Proof.* Let $u \notin C$. Domination gives at least one code neighbor. If $u$ had exactly one such neighbor $c$, then $I_C(u) = \{c\}$. Independence also gives $I_C(c) = \{c\}$. This contradicts separation of the edge $uc$. Thus every vertex outside $C$ has at least two code neighbors. Counting edges between $C$ and its complement yields

$$6|C| \ge 2(64 - |C|),$$

whence $8|C| \ge 128$ and $|C| \ge 16$. □

**Lemma 4.** *Every local 1-identifying code in $Q_6$ of size less than 16 contains a codeword with two distinct code neighbors.*

*Proof.* Suppose every vertex of the graph induced by $C$ has degree at most one. If this graph contains an edge $ab$, then $a$ and $b$ have no other code neighbors. Consequently $I_C(a) = I_C(b) = \{a, b\}$, contradicting (2). The induced graph must therefore have no edges, so $C$ is independent. Lemma 3 then gives $|C| \ge 16$. □

The two neighbors in Lemma 4 are nonadjacent: distinct neighbors of a cube vertex differ in two coordinates. Thus the three codewords induce a path on three vertices.

**Lemma 5** (Path normalization)**.** *If a local 1-identifying code in $Q_6$ has size at most 13, then a code of the same size exists that contains the integer vertices $0, 1, 2$.*

*Proof.* By Lemma 4, choose codewords $c, c \oplus e_i, c \oplus e_j$ with $i \neq j$. Translation $v \mapsto v \oplus c$ sends these to $0, e_i, e_j$. A coordinate permutation sends $e_i, e_j$ to $e_0, e_1$, represented by integers 1 and 2. Both operations are bijections preserving Hamming distances, closed balls, domination, and inequality of signatures. They preserve cardinality as well. □

This proves exactly the additional units used below. We impose no unproved restriction on the number of code components, the number of edges, or the distribution of codeword weights.

# 5 The exact Boolean reduction

For $v = 0, \ldots, 63$, let $x_{v+1}$ indicate whether $v$ belongs to $C$. Use the following clauses:

$$\bigvee_{c \in B_1(v)} x_{c+1} \qquad (v \in V(Q_6)), \tag{4}$$

$$\bigvee_{c \in B_1(u) \triangle B_1(v)} x_{c+1} \qquad (uv \in E(Q_6)), \tag{5}$$

$$x_1, \quad x_2, \quad x_3. \tag{6}$$

There are 64 clauses in (4) and 192 in (5). Domination is equivalent to the first family. The second family is equivalent to local separation since

$$I_C(u) \triangle I_C(v) = (B_1(u) \triangle B_1(v)) \cap C.$$

An adjacent pair in $Q_6$ has exactly its two endpoints in the intersection of its closed balls. Accordingly each domination clause has seven literals and each separation clause has ten. The normalization units are justified by Lemma 5.

## 5.1 The cardinality extension

To exclude size at most 13 we append a sequential counter. It uses symbols

$$s_{i,j} \qquad (1 \leq j \leq 13, \quad j \leq i \leq j + 50).$$

There are $13 \cdot 51 = 663$ auxiliary variables. The four clause families are

$$\neg x_i \vee s_{i,1} \qquad 1 \leq i \leq 51, \tag{7}$$

$$\neg s_{i,j} \vee s_{i+1,j} \qquad 1 \leq j \leq 13, \quad j \leq i \leq j + 49, \tag{8}$$

$$\neg x_{i+1} \vee \neg s_{i,j} \vee s_{i+1,j+1} \qquad 1 \leq j \leq 12, \quad j \leq i \leq j + 50, \tag{9}$$

$$\neg x_{i+1} \vee \neg s_{i,13} \qquad 13 \leq i \leq 63. \tag{10}$$

The clause counts are respectively 51, 650, 612, and 51, totaling 1 364. The actual DIMACS identifiers 65 through 727 are a bijective numbering of these symbols, checked against the persisted formula. The four displayed families describe their meaning independently of that numbering.

**Lemma 6** (Satisfying extension)**.** *For every assignment to $x_1, \ldots, x_{64}$ with $\sum_{k=1}^{64} x_k \leq 13$, the counter clauses admit a satisfying auxiliary assignment.*

*Proof.* Set $s_{i,j}$ to true precisely when $\sum_{k=1}^{i} x_k \geq j$. An input set to true contributes at least one to its prefix, proving (7). Prefix counts are monotone, proving (8). A true next input raises a prefix count by one, proving (9). Finally, a violation of (10) would mean that a prefix contains at least 13 true inputs and its next input is also true. This contradicts the assumed total bound. □

Only this direction is needed: every hypothetical small code must extend to a satisfying assignment of the actual formula. The lower-bound argument does not assume the correctness of a SAT library's cardinality encoder, nor does it need a converse theorem for arbitrary auxiliary assignments.

Let $F_{13}^{\mathrm{path}}$ denote the conjunction of (4)–(10). It has 727 variables and

$$64 + 192 + 3 + 1364 = 1623$$

clauses. An independent comparison reconstructs the graph clauses directly from Hamming distance and checks all counter clauses against the four families.

**Proposition 7.** *If $F_{13}^{\mathrm{path}}$ is unsatisfiable, every local 1-identifying code in $Q_6$ has size at least 14.*

*Proof.* A counterexample of size at most 13 normalizes by Lemma 5 to a code containing $0, 1, 2$. Its membership indicators satisfy the graph clauses and the units. Lemma 6 supplies values for every auxiliary variable, giving a model of $F_{13}^{\mathrm{path}}$, a contradiction. □

## 6 A finite contradiction certificate

The persisted file `exclude13-path.cnf` is $F_{13}^{\mathrm{path}}$. A SAT run and DRAT-to-LRAT conversion produced `exclude13-path.lrat`. Its additions are all reverse-unit-propagation (RUP) steps; no resolution-asymmetric-tautology (RAT) step is needed. The checked certificate contains 61 970 additions, 1 987 090 hint references, and 63 517 deleted clause references. Its final added clause, with identifier 92061, is empty.

For clarity, an RUP check of a proposed clause $D$ starts by assuming every literal of $D$ false. It follows an ordered list of already available clauses. A cited clause must be unit under the current partial assignment, thereby forcing its last unassigned literal, or false, thereby closing the contradiction. A contradiction establishes that the current formula entails $D$. Repeating this argument for every addition and deriving the empty clause proves unsatisfiability. Deletions only remove usable premises and cannot create a false entailment.

A standard-library-only Python checker implements these rules, checks that cited clauses are active, and accepts only positive LRAT hint identifiers. Its accepted result is therefore an exact finite derivation, not a numerical tolerance or an unverified solver status. The official `lrat-check` checker and `drat-trim` also accept the path certificate according to the retained run records. They provide separate software checks; they are not Lean kernel proofs.

There is also a longer certificate using only $x_1$ as a normalization unit. Translation by any codeword justifies that unit because domination implies $C \neq \varnothing$. This alternative has 1 621 input clauses and 343 158 RUP additions. It excludes small codes without the path-normalization lemma and has been checked independently. The smaller path route is the one selected for the final Lean assembly.

*Proof of Theorem 1 at the certificate level.* The exact certificate check establishes that $F_{13}^{\mathrm{path}}$ is unsatisfiable. Proposition 7 gives the lower bound 14; Proposition 2 gives the matching upper bound. □

# 7 Lean formalization and its trust boundary

The development targets Lean 4.24.0 and represents a vertex as `Fin 64`, a code as `Fin 64 -> Bool`, and its cardinality as the length of the filtered list of all vertices. Adjacency tests whether the integer XOR belongs to $\{1, 2, 4, 8, 16, 32\}$. A signature is a Boolean-valued function on all vertices, so signature inequality is literal extensional inequality, not an informal surrogate.

The checked semantic development supplies the following components:

1. The explicit witness, its cardinality, and equivalence between signature separation and hitting a neighborhood symmetric difference.
2. Translation and coordinate-permutation invariance, the independent-code lower bound, and normalization of every hypothetical size-at-most-13 code to contain $0, 1, 2$.
3. The concrete graph clauses, the prefix-count interpretation of every auxiliary variable, and satisfaction of every actual counter clause under the size bound.
4. A construction of the concrete input-clause proposition from any normalized small code.

These culminate in a conditional theorem: a refutation of the actual input proposition implies the exact minimum. In the source it is named

```
LocalIdentifyingQ6.minimum_fourteen_of_path_refutation
```

and its argument is, in ASCII notation,

```
refute : forall p : Nat -> Prop, Q6RUPPath.Inputs p -> False
```

Here `Inputs p` is an explicitly defined collection of ordinary proposition-valued records containing the 1 623 input clauses. It is not a new axiom. The theorem's conclusion is the following conjunction, also transcribed in ASCII:

```
(exists C : Code, LocalIdentifying C /\ cardinality C = 14) /\
(forall C : Code, LocalIdentifying C -> 14 <= cardinality C)
```

The semantic theorem isolates the finite refutation from the combinatorial reduction. Its `refute` argument is discharged by the complete RUP replay below, producing an unconditional theorem with no certificate or unsatisfiability hypothesis. Both the semantic theorem and the final theorem report only the standard axioms `propext`, `Classical.choice`, and `Quot.sound`.

## 7.1 Proof-producing RUP replay

The replay engine reads the RUP data and constructs ordinary Lean proof terms. Clauses are propositions built from disjunction, negation, and `False`. Unit propagation is implemented by disjunction elimination: already refuted literals are eliminated, leaving the forced literal or contradiction. Negating the target clause supplies the initial assumptions; classical contradiction converts the resulting refutation into a proof of the target clause. Each added theorem is submitted to Lean's declaration checker. Earlier derived clauses are cited by theorem name.

The metaprogram, JSON parser, solver, certificate converter, and Python generator are thus intended to be *proof producers*, not trusted logical oracles. A bug in them should cause rejected terms or a failure to build, rather than justify an invalid theorem. Finite semantic computations use kernel-checked decision proofs. The checked final dependency chain does not use `native_decide`, an external satisfiability axiom, or a `sorry` placeholder.

The replay is split across 194 modules to control elaboration and memory use: the first 87 contain 100 steps each, followed by modules with at most 500 steps. The final module is `RUPPathFast.Chunk0193`. After this chain, `FullTheorem.lean` applies the semantic theorem to `Q6RUPPath.c92061`. The resulting unconditional theorem is named

```
LocalIdentifyingQ6.minimum_local_identifying_Q6
```

The complete proof and independent clean-source rebuild both succeeded. The final conclusion is exactly the conjunction displayed above: a valid code exists with cardinality 14, and every valid code has cardinality at least 14. There is no additional refutation hypothesis in the final theorem.

### 7.2 Independent clean verification

The independent clean check compiled the exact final dependency closure: 451 source and replay-data files across 257 Lean modules, including all 194 RUP replay modules and `FullTheorem`. Every module completed with return code zero, and the final theorem's axiom report listed only the three standard axioms above. The source manifest, build order, per-module results, status, and complete log are retained in the accompanying artifact under `verification/clean/`.

The check copied the source and replay data into a new directory, used that directory as the project import path, and compiled in dependency order. No previously generated project `.olean` files were reused. The recorded run took approximately 2 128 seconds; the largest per-module peak resident memory was 4 135 660 KiB. These are observations for this run, not portable performance guarantees.

The trust boundary includes Lean's kernel implementation, the official toolchain and imported standard library, the execution platform, and preservation of the checked files. The official Lean and standard-library artifacts remain toolchain dependencies. Formal checking does not establish literature novelty or human agreement with the chosen definition.

## 8 Reproducibility and artifact identity

All paths below are relative to the accompanying artifact root. The exact primary formula and certificate have the following SHA-256 digests:

```
exclude13-path.cnf
13bcf1c4d71a670f0b4f7e8c12ddefd2115d36ac71257e9cd0b9d8e9752b6a99

exclude13-path.lrat
f554bf730712d5339fa9b4e15436161de6b58e81086d4ebd0e47d734c33328ca
```

These identify the input formula and contradiction certificate. The clean-check manifest of all 451 source and replay-data files has SHA-256 digest

```
0db2b23683bd596b65975841a75677ba05dc92fafba7f87fe03b40e87ed0c566
```

Toolchain: Lean 4.24.0, commit `797c613eb9b6d4ec95db23e3e00af9ac6657f24b`. The source-closure digest and certificate digests identify distinct objects; a matching hash establishes identity,

while the corresponding verification establishes validity.

The release stores large certificates compressed. The first command below verifies their recorded digests and materializes the raw files; the remaining commands rerun independent checks without repeating the optimization search:

```
python3 materialize_certificates.py
python3 independent_witness_check.py
python3 independent_encoding_check.py
python3 independent_path_encoding_check.py
python3 independent_lrat_check.py \
  exclude13-path.cnf exclude13-path.lrat
python3 independent_replay_data_check.py
sha256sum exclude13-path.cnf exclude13-path.lrat
```

The first checker uses direct Hamming distances. The encoding checks validate the original counter and the path instance's only graph change: replacing the single unit $x_1$ with the three units $x_1, x_2, x_3$. The replay-data check compares concrete generated input propositions and RUP data with the certificate, including the final empty clause. This comparison is valuable audit evidence but is not a substitute for Lean checking the proof terms.

With the official `lrat-check` binary built from the retained `drat-trim` source, run

```
lrat-check exclude13-path.cnf exclude13-path.lrat
```

The retained source revision is

```
2e3b2dc0ecf938addbd779d42877b6ed69d9a985
```

from the upstream `drat-trim` repository [2]. For a new clean Lean check, use a *previously nonexistent* destination:

```
LEAN=/path/to/lean-4.24.0/bin/lean \
  python3 verify_lean_clean.py /tmp/q6-clean-verification
```

The script refuses to overwrite an existing destination. Completion is recorded in `status.json`; `success: true` is required. The log, source manifest, build order, and per-module results are retained alongside it. Checking partial progress or merely finding output files is insufficient. The full replay can be substantially more expensive than the independent RUP checker, so no unmeasured runtime or memory guarantee is made.

The discovery optimization and SAT search are separable from verification. In particular, the floating-point mixed-integer optimization result is useful for finding the witness but is not a premise of the exact lower-bound argument. Reproducing the finite verification of persisted artifacts is the shortest route to assessing the reported result.

## 9 Limitations and further questions

This result determines a single finite parameter. It does not classify all 14-word codes, count their equivalence classes under cube automorphisms, or give a formula for other dimensions. Although the structural reduction is elementary, the final exclusion uses a substantial propositional derivation rather than a short human-only proof.

# A A compact exhaustive witness check

The following Python program is self-contained. Python integers are exact, and the loops enumerate all vertices and all unordered adjacent pairs.

```
C = {0, 4, 15, 23, 26, 29, 31,
     33, 35, 36, 41, 49, 58, 62}

def distance(u, v):
    return (u ^ v).bit_count()

I = [{c for c in C if distance(v, c) <= 1}
     for v in range(64)]
assert len(C) == 14
assert all(I)
edges = [(u, v) for u in range(64)
         for v in range(u + 1, 64)
         if distance(u, v) == 1]
assert len(edges) == 192
assert all(I[u] != I[v] for u, v in edges)
print("Verified: 14 codewords, 64 vertices, 192 edges")
```

This check is intentionally independent of any SAT encoding or optimization package. It proves the upper-bound claim as an exhaustive finite software calculation; the accompanying Lean theorem provides a kernel-checked version of that calculation.

# B Evidence ledger

| Claim | Evidence and status |
|---|---|
| Size-14 witness | Independent exhaustive checker; Lean witness and cardinality theorems checked. |
| Every small code gives input clauses | Elementary arguments in Sections 4–5; checked Lean semantic bridge, including path normalization. |
| Input formula is unsatisfiable | Exact RUP checker and official certificate tools accept the stored LRAT proof. |
| Complete kernel-checked exact minimum | Full replay and unconditional theorem passed, with standard axioms only. |
| Independent clean-source reproduction | All 257 modules passed from the pinned 451-file source closure. |
| Improvement over located prior work | Prior upper bound 15 verified in the primary 2024 paper; no earlier 14 found in the scoped review. |
| Global priority or human peer review | Not established. |

# References


[1] V. Herva, T. Laihonen, and T. Lehtilä. *Optimal local identifying and local locating-dominating codes.* Fundamenta Informaticae **191** (2024), 351–378. DOI: 10.3233/FI-242187. Journal PDF: https://fi.episciences.org/13881/pdf. See Definition 1.3, Table 1, and Proposition 2.9. Versioned preprint: https://arxiv.org/abs/2302.13351v3.

[2] M. J. H. Heule and contributors. *drat-trim*, source repository and certificate checkers. https://github.com/marijnheule/drat-trim. The source revision is recorded in Section 8.

[3] Lean contributors. *Lean 4*, version 4.24.0. https://github.com/leanprover/lean4/releases/tag/v4.24.0. The toolchain version is pinned by the accompanying `lean-toolchain` file.